ISTA 130

Classes I

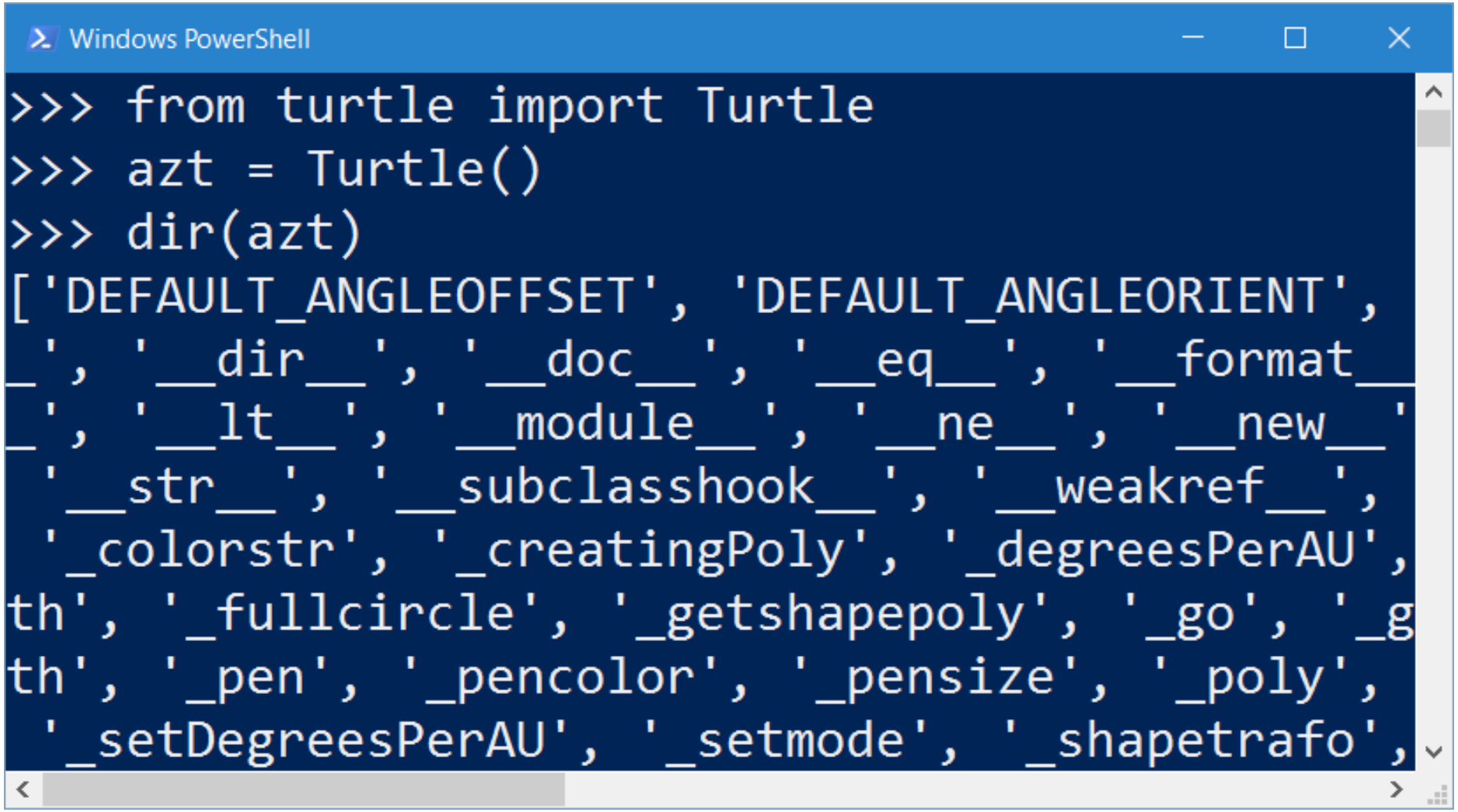
Last Time

- Some more dictionaries.
- Mutable vs. Immutable.

Our Programming Toolkit.

- `if` statements, `for` and `while` loops.
- Strings and lists (sequence types): creating, indexing, slicing.
- Traversing a string or list by index or by element. When must you traverse by index? When can you traverse by element?
- Traverse a dictionary by key.
- Open and traverse a file by line. Write to a file.
- Going back and forth between lists and strings, lists and dictionaries.
- Other string, list, and dictionary functions and methods.

Memory Lane…


Windows PowerShell
>>> from turtle import Turtle
>>> azt = Turtle()
>>> dir(azt)
['DEFAULT_ANGLEOFFSET', 'DEFAULT_ANGLEORIENT',
_', '__dir__', '__doc__', '__eq__', '__format__
_', '__lt__', '__module__', '__ne__', '__new__'
'__str__', '__subclasshook__', '__weakref__',
'_colorstr', '_creatingPoly', '_degreesPerAU',
th', '_fullcircle', '_getshapepoly', '_go', '_g
th', '_pen', '_pencolor', '_pensize', '_poly',
'_setDegreesPerAU', '_setmode', '_shapetrafo',

Book Example

Imagine we're writing software to keep track of books for a used bookstore:

- We'd like to keep track of the following information for each book we want to sell in our store:
  - title
  - author
  - price
  - number in stock
- We could use lists and dictionaries to keep track of all of this information, e.g:
  - 4 lists to hold titles, authors, prices, and quantities?
    - we'd have to be careful to always coordinate the index numbers across the lists or we'd lose information
  - a dictionary mapping title to a list of [author, price, quantity]
  - many other ways

Defining New Types

Wouldn't it be nice if there were a built in `Book` type in `Python`?

We might use it like this:

```
a_book = Book('The Hobbit', 'Tolkien')
```

`print(a_book.title)` → `The Hobbit`

`print(a_book.author)` → `Tolkien`

`print(a_book.price_for_copies(3))` → `32.97`

- There is no built in `Book` type, but we can define our own!

Defining New Types

In order to define a new `Python` type we create a **class**:

- A *class* is a way to group data and methods together into a convenient unit (object)
  - (`Turtle` is a class)
- To design a class, we'll need to think about:
  - the *state* that each instance will keep track of about itself:
    - (each Turtle instance keeps track of its own pencolor, pensize, speed, etc.)
  - the kinds of *methods* the new class of objects will need:
    - (`forward`, `right`, `penup`, etc.)

Some English Definitions

class:

a number of persons or things regarded as forming a group by reason of common *attributes*, characteristics, qualities, or traits

- e.g. dogs, fruit, cars, ...

instance:

a case or occurrence of anything

- e.g. Snoopy, Benji, the pineapple in my fridge, my friend's sister's blue 1971 Gremlin, ...
- we distinguish the different instances by their attributes

method:

a procedure, technique, or way of doing something

- e.g. how to feed a dog, how to slice a pineapple, how to parallel park a car

Python Classes

In Python we will:

- define new classes
  - you can think of a class as a blue-print for the construction of some type of object
  - a class definition is similar to a function definition in that Python doesn't do anything with it until we write code that uses it
- construct instances (objects) from classes
  - Python reads the blue-print and constructs one of the objects described by the blue-print
  - (e.g. `yertle = turtle.Turtle()`)
- apply methods to particular instances
  - perform some procedure using a particular object
  - (e.g. `yertle.forward(100)`)

Class `Book`

The class definition begins with a header:

class `<classname>:`

- (this defines a new *type*)

- A **class** definition is a compound statement (like **def**, **while**, **if**, ...)
  - The name of your class should begin with a capital letter:

    class `Book:`

- We create an *instance* by *calling* the class:

  `my_book = Book()`

  - We'll refer to this as calling the *constructor* function: it constructs new instances

Class `Book`

- Each new book instance that we create needs its own title
  - we'd like to put it in a variable named `title`
  - each time we construct a new Book instance we'd like to tell `Python` the new book's title
- To do this we define a method called **__init__** *inside* of our `class`
- The **__init__** method allows us to specify the initial contents of each new object we create
  (i.e. we **init**ialize each instance)

Class Book

We define an `__init__` method *inside* our class like this:

```
def __init__(self, <param1>, <param2>, ...):
    # method body here
```

- The first parameter to an **__init__** method will ALWAYS be **self**
- The **self** parameter refers to the particular instance we are in the process of creating.

Class `Book`

For our Book class **__init__** will look like this:

```
def __init__(self, the_title):
    self.title = the_title
    return None
```

- We see the dot-operator applied to `self`

  self.title = the_title

- Clearly we're assigning the value of `the_title` to something, but what?
- If the statement looked like this:

  ```
  x = the_title
  ```

  we'd know it meant to create a new variable called `x` and assign the value of the `the_title` to `x`

Class `Book`

```
def __init__(self, the_title):
    self.title = the_title
    return None
```

- The **self** parameter will automatically refer to the new unborn Book *instance*
- **self.title** indicates a variable named `title` found inside the Book instance being constructed:
  - we're giving *each* new Book its *own* variable called `title`
  - and assigning it the value of the parameter `the_title`
  - think of the dot-operator as a possessive form, e.g: `self.title` means `self`'s `title` variable
  - we'll refer to such a variable as an *attribute* or an *instance variable*

Class `Book`

- When we put an `__init__` method in our class:
  - We always give it **self** as the first parameter
- When we call the `Book` class, `Python` creates a *new* Book instance
- `Python` then calls the `__init__` method passing it the new instance and any arguments we pass to `Book`
- Finally `Python` returns the *new* Book instance
- With our new `__init__` method, when we create a Book we do it like this:

```
my_book = Book("The Hobbit")
```

Behind The Scenes

- We don't get to see what Python is doing behind the scenes when we do this:

```
my_book = Book("The Hobbit")
```

- But we can *imagine* we're calling a function that looks like this:

```
def Book(the_title):
    new_object = magically_create_a_new_object()
    __init__(new_object, the_title)
    return new_object
```

- (i.e. Python passes the newly created object to our `__init__` method)

Class Book

```
def __init__(self, the_title):
    self.title = the_title
    return None
```

- Our `__init__` method clearly takes 2 parameters, but we only pass 1 argument to the `Book` constructor
  - `Python` takes care of passing `self`
  - We don't call the `__init__` method directly
    - we call the Book constructor
    - `Python` uses our `__init__` method to help it create the new Book instance
- The `Book` function returns the newly created instance.

Improving The `Book` Class

To make the Book more useful we'll add more *attributes* (also known as *instance variables*):

- add an attribute for the **author**
- add an attribute for the **price**
- add an attribute for the **quantity** in stock
- The `__init__` method's parameters are different variables from the instance variables
  - It's common practice to give them the same name

    ```
    self.title = title
    ```

  - Don't let this confuse you

    `title` is the **parameter**

    `self.title` is the **instance's own variable** by the same name!

Improving The `Book` Class

What happens when we `print` a Book instance?

- `Python` by default prints something that looks pretty weird
  - it tells us we're printing a Book object (and shows us a memory address)
  - `Python` can't know on its own how we would like Book objects to be printed
- To let `Python` know how to print Book objects we'll define another method inside the Book class:

```
def __repr__(self)
```

QUICK VOCABULARY REVIEW

- **class**: for us, a user-defined type. `Turtle` is a class. In Python, all types are classes.
- **object/instance** : an individual example of a class. `tallulah` is an instance of the `Turtle` class, also called a `Turtle` object.
- **method** : a special type of function that is called with syntax `object.method()`